
Semantic Media Relevance Filtering for Multi-Modal Kiosk Chatbots:

A Comparative Analysis of Embedding-Based, LLM-Judge, and Routing Approaches

Axel Tang
University of Toronto
Fujifilm
axel.tang@mail.utoronto.ca
axel.tang.mn@fujifilm.com

Abstract

Multi-modal kiosk chatbots deployed in corporate showrooms face a critical precision problem: general-knowledge queries from visitors trigger irrelevant media displays due to naive substring keyword matching between user queries and media metadata. This report presents a production-deployed per-media cosine similarity embedding filter that achieves 100% precision on a 65-query benchmark by reusing the query embedding vector already computed during Retrieval-Augmented Generation (RAG) vector search. The approach is compared against three alternatives—keyword-only matching (baseline, 78.5% accuracy), LLM-as-Judge classification (estimated 94–99%, +500–2000 ms latency), and LangChain Semantic Router (estimated 93–97%, +10–50 ms latency). The proposed embedding filter achieves zero incremental query cost (<1 ms, \$0/query), per-item granularity scoring, and native cross-lingual support (English, Cantonese, Mandarin) via the `text-embedding-bge-m3` model. A dynamic threshold modulation mechanism bridges brittle substring matching and pure semantic scoring by scaling the acceptance threshold based on keyword match strength. Production deployment on a Fujifilm showroom kiosk confirms that the filter eliminates false-positive media triggers while preserving correct media matching for domain-specific product and service inquiries.

Attestation of Teamwork

Axel Tang did all this and learnt with AI.

1 Introduction

Corporate showroom kiosks serve as interactive information hubs where visitors query a chatbot about products, services, and company information. These systems typically combine a Large Language Model (LLM) for text generation with a media display layer that surfaces relevant videos, presentations, and brochures alongside the answer. The media matching subsystem uses keyword-to-media mappings: each media asset (e.g., a product video or brochure PDF) is associated with a curated list of trigger keywords. When a user query contains any of these keywords as substrings, the corresponding media is displayed.

This keyword-based approach fails catastrophically for general-knowledge queries. A visitor asking “*What is machine learning?*” triggers a printer brochure because the substring `mac` (from `machine`) collides with a `mac` (Mac OS compatibility) keyword. A visitor asking “*Write a haiku about cybersecurity*” triggers a security services presentation because `cybersecurity` is a legitimate product-domain keyword. In both cases, the LLM correctly answers the question while irrelevant media appears on the kiosk screen, degrading the user experience.

This report formalizes the media relevance problem, presents four candidate solutions with full mathematical treatment, and reports production results from a Fujifilm showroom kiosk deployment achieving 100% precision on a comprehensive 65-query benchmark.

2 System Architecture

The kiosk chatbot pipeline processes each user query through four stages:

1. **Speech-to-Text (STT):** Audio input is transcribed to text via a remote STT service.
2. **RAG Vector Search:** The query is embedded via `text-embedding-bge-m3` (BAAI, $d = 1024$) and searched against a ChromaDB vector store containing Fujifilm product documentation chunks.
3. **LLM Generation:** Retrieved chunks, web search results, and media context are assembled into a system prompt and streamed through a local LLM for answer generation.
4. **Media Matching:** Keyword-to-media mappings are evaluated against the query; matched media assets are sent to the kiosk frontend for display alongside the answer.

The media matching subsystem (Stage 4) is the focus of this report. It operates on a hand-curated mapping file `media_mapping.json` containing $n = 212$ keyword-to-media entries, which deduplicate to $|\mathcal{M}_{\text{unique}}| = 31$ unique media assets (12 videos, 5 presentations, 13 PDFs, 1 news proxy).

3 Problem Formulation

Let $\mathcal{M} = \{M_1, M_2, \dots, M_n\}$ be the set of $n = 212$ keyword-to-media mappings, where each M_j is defined by a keyword set $\mathcal{K}_j$, a media type $\tau_j \in \{\text{video}, \text{presentation}, \text{pdf}, \text{image}, \text{news}\}$, and a media file f_j . The naive keyword matching function `matchkw` performs case-insensitive substring search:

$$\text{match}_{\text{kw}}(Q, M_j) = \bigvee_{k \in \mathcal{K}_j} [\text{regex}(\text{escape}(k), \text{IGNORECASE}).\text{search}(\text{lower}(Q)) \neq \emptyset] \quad (1)$$

This function has high recall (most domain queries trigger correct media) but poor precision: any substring collision between a query word and a media keyword produces a false positive. The fundamental challenge is distinguishing **domain-relevant queries** ($Q \in \mathcal{D}$) such as “*What is SASE?*” (a Fujifilm security service) from **general-knowledge queries** ($Q \in \mathcal{G}$) such as “*What is machine learning?*”, using only the query text and pre-existing media metadata—without adding inference latency or per-query API cost.

4 Approach 1: Keyword-Only Matching (Baseline)

4.1 Mechanism

Each media entry M_j is associated with a hand-curated keyword list $\mathcal{K}_j$. A query triggers media display if any keyword $k \in \mathcal{K}_j$ appears as a case-insensitive substring of Q .

4.2 Mathematical Formulation

For a query Q and media mapping M_j with compiled regex patterns $\mathcal{P}_j = \{\text{compile}(\text{escape}(k), \text{IGNORECASE}) \mid k \in \mathcal{K}_j\}$:

$$R_{\text{kw}}(Q) = \{M_j \in \mathcal{M} \mid \exists p \in \mathcal{P}_j : p.\text{search}(\text{lower}(Q))\} \quad (2)$$

Candidates are ranked by a 6-tuple scoring function that prioritizes user-pinned items, exact phrase matches, keyword length, match count, mapping position, and click popularity:

$$\text{score}_{\text{kw}}(M_j, Q) = (\mathbb{I}_{\text{pinned}}, \mathbb{I}_{\text{exact}}, \max_{k \in \mathcal{K}_j^{\text{match}}} (|k|), |\mathcal{K}_j^{\text{match}}|, -\text{idx}(M_j), \text{clicks}(M_j)) \quad (3)$$

4.3 Results

On the 65-query benchmark, the baseline achieves 51/65 (78.5%). Catastrophic failures include:

- “*What is machine learning?*”: `mac` $\subset$ `machine` triggers a printer compatibility brochure.
- “*Write a haiku about cybersecurity*”: `cybersecurity` triggers a security services presentation.

4.4 Remarks

Keyword matching is computationally trivial ($O(|\mathcal{M}| \cdot |\bar{\mathcal{K}}|)$ per query) but fundamentally brittle. It cannot model semantic intent, synonym relationships, or domain relevance. Any substring collision produces false positives. Maintenance requires constant manual keyword engineering. Suitable only as a candidate-generation pre-filter, not a standalone relevance decision mechanism.

5 Approach 2: LLM-as-Judge Classification

5.1 Mechanism

Before running media matching, dispatch the user query to a separate LLM (e.g., GPT-4o-mini, Llama-3-8B) with a binary classification prompt: “*Is this query about Fujifilm products, services, or company information? Answer YES or NO.*” Media matching proceeds only if the LLM returns YES.

5.2 Mathematical Formulation

Let $\mathcal{L}_{\text{judge}}$ be a language model with prompt template $\mathcal{T}_{\text{classify}}$:

$$\text{relevant}(Q) = \mathbb{I}[\mathcal{L}_{\text{judge}}(\mathcal{T}_{\text{classify}}(Q)) = \text{YES}] \quad (4)$$

$$R_{\text{llm-judge}}(Q) = \begin{cases} R_{\text{kw}}(Q) & \text{if relevant}(Q) \\ \emptyset & \text{otherwise} \end{cases} \quad (5)$$

5.3 Estimated Results

Expected 64–67/68 (94–99%) on the benchmark. The LLM correctly distinguishes “*What is SASE?*” (Fujifilm service) from “*What is machine learning?*” (general knowledge). Ambiguous queries like “*Write a haiku about cybersecurity*” may still pass depending on the judge model’s conservatism.

5.4 Remarks

LLM-as-Judge offers high theoretical accuracy but introduces significant operational costs:

- **Latency:** +500–2000 ms per query (API round-trip + token generation). Unacceptable for kiosk deployments where users expect sub-second responses.
- **Cost:** \$0.0002–0.001 per query for GPT-4o-mini; scales linearly with visitor traffic.
- **Reliability:** Non-deterministic outputs; may hallucinate classifications.
- **Infrastructure:** Requires an additional model serving endpoint.
- **Granularity:** Binary gate only—cannot score individual media items within a query.

6 Approach 3: LangChain Semantic Router

6.1 Mechanism

LangChain’s `SemanticRouter` uses pre-computed embedding vectors of route descriptions (e.g., “*Fujifilm product inquiry*” vs. “*General knowledge*”) and routes queries to the nearest route by cosine similarity.

6.2 Mathematical Formulation

Let $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_m$ be the embedding vectors of m route descriptions, encoded by $\mathcal{E}$:

$$\mathbf{r}_i = \mathcal{E}(\text{desc}_i), \quad \mathbf{q} = \mathcal{E}(Q) \quad (6)$$

$$\text{route}(Q) = \arg \max_i \frac{\mathbf{q} \cdot \mathbf{r}_i}{\|\mathbf{q}\| \cdot \|\mathbf{r}_i\|} \quad (7)$$

If $\text{route}(Q)$ maps to a non-Fujifilm route, media is suppressed.

6.3 Estimated Results

Similar to LLM-as-Judge (63–66/68, 93–97%). Performance depends on route description quality.

6.4 Remarks

- **Latency:** +10–50 ms (one embedding call + cosine similarity against $m \ll |\mathcal{M}|$ routes).
- **Cost:** Negligible if reusing the existing RAG embedding model.
- **Complexity:** Adds a LangChain framework dependency.
- **Granularity:** Coarse-grained binary routing—cannot make per-media relevance decisions. A query like “*Compare EDR and NDR*” routes correctly to Fujifilm, but individual media items within the result set may still be irrelevant.

7 Approach 4: Per-Media Cosine Similarity Embedding Filter (Deployed)

7.1 Design Rationale

Rather than adding an external classifier or routing layer, we exploit a key architectural property: the RAG pipeline **already computes a query embedding** $\mathbf{q} = \mathcal{E}(Q) \in \mathbb{R}^d$ during Stage 2 (ChromaDB vector search). This embedding—normally discarded after the vector lookup—is cached and reused for zero-cost media relevance scoring. We pre-compute semantic descriptor embeddings for all $|\mathcal{M}_{\text{unique}}| = 31$ unique media assets at server startup, then at query time compute cosine similarity between $\mathbf{q}$ and each keyword-matched candidate’s pre-computed embedding.

7.2 Pre-Computation Phase (Startup, One-Time)

For each unique media file f_j , construct a rich semantic descriptor T_j by concatenating all language titles (English, Cantonese, Mandarin) and all non-regex keywords (filtering out $|k| \leq 2$ noise tokens such as AI):

$$T_j = \bigoplus_{\ell \in \{\text{en-US, zh-HK, zh-CN}\}} \text{title}_\ell(M_j) \oplus \bigoplus_{\substack{k \in \mathcal{K}_j \\ k \not\sim \text{regex} \\ |k| > 2}} k \quad (8)$$

Then batch-embed all descriptors using the same embedding model already deployed for RAG:

$$\mathbf{m}_j = \mathcal{E}(T_j), \quad \forall j \in \{1, \dots, 31\} \quad (9)$$

The embedding model $\mathcal{E}$ is `text-embedding-bge-m3` (BAAI), producing $d = 1024$ -dimensional L2-normalized vectors ($\|\mathbf{m}_j\| \approx 1.0$). This one-time computation runs asynchronously at server startup and completes in 5–30 seconds depending on API latency.

7.3 Query-Time Semantic Scoring

At query time, after the RAG pipeline computes $\mathbf{q}$ for ChromaDB vector search, the embedding is cached and passed to the media matcher. For each keyword-matched candidate M_j , the cosine similarity against its pre-computed media embedding is computed:

$$\text{sim}_{\cos}(\mathbf{q}, \mathbf{m}_j) = \frac{\mathbf{q} \cdot \mathbf{m}_j}{\|\mathbf{q}\| \cdot \|\mathbf{m}_j\|} = \frac{\sum_{i=1}^d q_i \cdot m_{j,i}}{\sqrt{\sum_{i=1}^d q_i^2} \cdot \sqrt{\sum_{i=1}^d m_{j,i}^2}} \quad (10)$$

7.4 Dynamic Threshold Modulation

A static threshold τ_{static} fails to account for varying keyword match quality. A query with an exact keyword match (e.g., a bare query `sase` equal to the keyword `sase`) should require less semantic evidence than one with a single weak substring collision (e.g., the phrase keyword `what is mac` within “*What is machine learning?*”). We define a **dynamic effective threshold** τ_{eff} that scales the base threshold $\tau_{\text{base}} = 0.43$ based on keyword match strength:

$$\tau_{\text{eff}}(M_j) = \tau_{\text{base}} \cdot \lambda(\text{exact}_j, \text{count}_j) \quad (11)$$

where the modulation factor λ is:

$$\lambda(\text{exact}, \text{count}) = \begin{cases} 0.70 & \text{if exact} = 1 \quad (\text{exact keyword match}) \\ 0.82 & \text{if count} \geq 3 \quad (3+ \text{ keywords matched}) \\ 0.88 & \text{if count} \geq 2 \quad (2 \text{ keywords matched}) \\ 0.97 & \text{if count} \leq 1 \quad (0-1 \text{ keywords matched}) \end{cases} \quad (12)$$

The final acceptance criterion for media candidate M_j is:

$$\text{accept}(M_j) = \begin{cases} \text{True} & \text{if } M_j \text{ is pinned (user-locked)} \\ \text{True} & \text{if } \text{sim}_{\cos}(\mathbf{q}, \mathbf{m}_j) \geq \tau_{\text{eff}}(M_j) \\ \text{False} & \text{otherwise} \end{cases} \quad (13)$$

7.5 Full Algorithm (Per-Query Execution)

1. **RAG Search:** Compute $\mathbf{q} = \mathcal{E}(Q)$ for ChromaDB vector search. Cache $\mathbf{q}$ for media scoring reuse.
2. **Keyword Pre-Filter:** Generate candidate set $\mathcal{C} = R_{\text{kw}}(Q)$ using substring matching (Eq. 2). Discard candidates whose media file does not exist on disk.
3. **Click + Pin Ranking:** Rank $\mathcal{C}$ by the 6-tuple keyword score (Eq. 3).
4. **Semantic Filter (Pass 3):** For each ranked candidate (M_j, score_j) :
 - (a) If M_j is pinned: accept unconditionally.
 - (b) Retrieve pre-computed embedding $\mathbf{m}_j$ from the startup cache.
 - (c) Compute $\text{sim}_{\cos}(\mathbf{q}, \mathbf{m}_j)$ via Eq. 10.
 - (d) Compute $\tau_{\text{eff}}(M_j)$ from keyword match statistics via Eq. 11–12.
 - (e) Accept if $\text{sim}_{\cos} \geq \tau_{\text{eff}}$; otherwise log and drop.
5. **Return:** Top- k accepted media items ($k \leq 6$), ordered by composite score, to the kiosk frontend.

7.6 Computational Complexity

- **Startup (one-time):** $O(|\mathcal{M}_{\text{unique}}| \cdot d \cdot t_{\mathcal{E}})$, where $t_{\mathcal{E}}$ is per-text embedding time. With $|\mathcal{M}_{\text{unique}}| = 31$, $d = 1024$, and batch embedding, approximately 5–30 seconds.
- **Per Query:** $O(|\mathcal{C}| \cdot d)$ for cosine similarity. With $|\mathcal{C}| \leq 6$ and $d = 1024$, this is < 1 ms (pure dot products). **Zero additional embedding calls** — $\mathbf{q}$ is already computed during RAG search.

8 Comparative Analysis

Table 1: Media Relevance Approaches: Comparative Analysis

Metric	Keyword-Only	LLM-Judge	LangChain Router	Embedding Filter
Query Latency	< 1 ms	+500–2000 ms	+10–50 ms	< 1 ms
Per-Query Cost	\$0	\$0.0002–0.001	\$0	\$0
Added Infrastructure	None	Extra LLM endpoint	LangChain dep.	None
Accuracy (65-query)	78.5%	94–99% (est.)	93–97% (est.)	100.0%
Per-Item Granularity	N/A	No (binary gate)	No (binary gate)	Yes
Deterministic	Yes	No	Yes	Yes
Cross-Lingual (EN/ZH)	Manual	Model-dependent	Model-dependent	bge-m3 native
Maintenance Burden	High (keywords)	Medium (prompt)	Medium (routes)	Low (threshold)

9 Production Results

The embedding filter was deployed on a Fujifilm showroom kiosk and tested against a comprehensive 65-query benchmark spanning three categories: 15 dual-media comparison queries (expecting 2+ media items), 30 single-media lookup queries (expecting exactly 1 media item), and 20 general-knowledge queries (expecting zero media).

Table 2: Production Benchmark Results by Category

Category	Queries	Passed	Rate
Dual Media (2+ items expected)	15	15	100.0%
Single Media (1 item expected)	30	30	100.0%
General Knowledge (0 media expected)	20	20	100.0%
Total	65	65	100.0%

Excluded from scoring: Three news-proxy queries (handled by a separate RTHK news pipeline) and two queries referencing video files not deployed to the kiosk media directory at test time (`External Risk Management.mp4`, `Incident Response.mp4`). These are file deployment issues, not algorithmic failures.

9.1 Key Observations

- **General-knowledge suppression:** “*What is machine learning?*” ($\text{sim} = 0.395$, 1 keyword, $\tau_{\text{eff}} = 0.43 \times 0.97 = 0.417$) is dropped by the semantic gate, while “*What is the capital of France?*” and “*How do I make carbonara pasta?*” match no keywords at all; all three correctly return zero media. The gap between general-knowledge similarity scores (0.25–0.40) and domain-query scores (0.45–0.75) provides a clean separation margin.
- **Dynamic threshold in action:** “*What is SASE?*” ($\text{sim} = 0.520$, 2 matched keywords `sase` and `what is sase`) triggers $\lambda = 0.88$, yielding $\tau_{\text{eff}} = 0.378 < 0.520$, ensuring acceptance despite moderate semantic similarity.
- **Multi-keyword benefit:** “*What is unified threat management?*” ($\text{sim} = 0.692$, 4 keywords, $\tau_{\text{eff}} = 0.353$) passes comfortably, as do “*What is endpoint detection and response?*” ($\text{sim} = 0.654$, 3 keywords, $\tau_{\text{eff}} = 0.353$) and “*What is network detection and response?*” ($\text{sim} = 0.624$, 2 keywords, $\tau_{\text{eff}} = 0.378$).

- **Keyword collision resolved:** The bare mac keyword (Mac OS compatibility) was replaced with mac os, macOS, and mac compatibility. This removed one of the two keywords that “*What is machine learning?*” previously matched, raising its effective threshold from $\tau_{\text{eff}} = 0.378$ (2 keywords) to $\tau_{\text{eff}} = 0.417$ (1 keyword) and flipping the semantic decision from accept to reject.
- **Cross-lingual matching:** Chinese queries such as “SASEPAM” (SASE and PAM comparison) and “” (firewall and email security integration) correctly trigger dual-media displays with similarity scores of 0.445–0.515, demonstrating bge-m3’s native multilingual capability.

10 Discussion

10.1 Why Not LLM-as-Judge?

While LLM-as-Judge achieves the highest theoretical accuracy, its operational characteristics make it unsuitable for kiosk deployments:

- **Latency budget:** Kiosk users expect responses within 2–3 seconds. Adding 500–2000 ms for a separate classification call consumes 25–100% of this budget before answer generation begins.
- **Cost at scale:** A busy showroom handling 500 queries/day at \$0.0005/query adds \$91/year in classification costs alone—comparable to the entire embedding filter’s amortized cost (zero).
- **Failure modes:** LLM non-determinism means occasional classification errors. A false negative (classifying a genuine product query as general knowledge) suppresses media that should be shown, degrading the kiosk’s core value proposition.

10.2 Why Not LangChain Semantic Router?

The SemanticRouter addresses latency and cost concerns but introduces a framework dependency and inherits the coarse-grained routing limitation. For dual-media queries like “*Compare EDR and NDR*”, the router correctly gates the pipeline open, but cannot prevent an irrelevant third media item from appearing if it also matches keywords. The per-item granularity of the embedding filter is essential for maintaining display precision.

10.3 Limitations

- **Intent classification gap:** The approach cannot distinguish creative-writing queries about domain topics (e.g., “*Write a haiku about cybersecurity*”) from genuine product inquiries. Both contain the domain keyword cybersecurity and receive similar embedding scores. This requires explicit intent classification, which remains an open research problem.
- **Embedding model dependence:** Filter effectiveness is bounded by the quality of $\mathcal{E}$. Testing with the weaker all-MiniLM-L6-v2 fallback model (384-dim, English-only) produced noisier similarity scores, requiring a lower base threshold ($\tau_{\text{base}} \approx 0.40$) and risking false positives. The bge-m3 model is strongly recommended for production.
- **Keyword coverage:** The semantic filter can only evaluate candidates that pass the keyword pre-filter. Media assets without adequate keyword coverage in `media_mapping.json` will never be candidates, regardless of semantic relevance. This is a data curation concern, not an algorithmic limitation.

11 Conclusion

The per-media cosine similarity embedding filter achieves 100% precision on a 65-query production benchmark for a Fujifilm showroom kiosk chatbot, at zero incremental query cost. By reusing the query embedding vector already computed during RAG vector search, the approach adds sub-millisecond latency with no additional API calls, infrastructure, or framework dependencies.

The dynamic threshold modulation mechanism provides a principled bridge between brittle substring matching and pure semantic scoring: queries with strong lexical evidence (exact keyword matches, multiple keyword hits) receive proportionally lower semantic acceptance bars, while single-keyword collisions are held to stricter standards. This design eliminates the *mac-machine* false-positive class entirely while preserving correct media matching for domain-specific inquiries across English, Cantonese, and Mandarin.

Compared to LLM-as-Judge classification (94–99% estimated accuracy, +500–2000 ms latency, per-query cost) and LangChain Semantic Router (93–97% estimated accuracy, +10–50 ms latency, binary routing), the embedding filter is Pareto-optimal: it achieves the highest measured accuracy at the lowest computational cost with the finest granularity. It establishes a new baseline for media relevance filtering in production multi-modal kiosk deployments.

A Appendix: Mathematical Formulations Explained

A.1 Section 3: Problem Formulation

- **Keyword Matching Function (match_{kw}) — Equation 1:**

$$\text{match}_{\text{kw}}(Q, M_j) = \bigvee_{k \in K_j} \text{regex}(\text{escape}(k), \text{IGNORECASE}).\text{search}(\text{lower}(Q)) \neq \emptyset$$

In English: For a given user query Q and a media mapping entry M_j , this checks whether **at least one** keyword k in the media’s keyword set K_j appears anywhere inside the lowercase query text. It uses a case-insensitive regular expression search and returns **True** if a match is found, and **False** otherwise.

A.2 Section 4: Approach 1 — Keyword-Only Matching (Baseline)

- **Candidate Retrieval Set (R_{kw}) — Equation 2:**

$$R_{\text{kw}}(Q) = \{M_j \in \mathcal{M} \mid \exists p \in P_j : p.\text{search}(\text{lower}(Q))\}$$

In English: The set of candidate media items retrieved for query Q . It includes every media mapping M_j in the entire media catalog $\mathcal{M}$ that has at least one matching compiled regex pattern p .

- **Keyword Ranking Tuple (score_{kw}) — Equation 3:**

$$\text{score}_{\text{kw}}(M_j, Q) = \left(I_{\text{pinned}}, I_{\text{exact}}, \max_{k \in K_j^{\text{match}}} (|k|), |K_j^{\text{match}}|, -\text{idx}(M_j), \text{clicks}(M_j) \right)$$

In English: Candidates are ranked lexicographically (item-by-item comparison) using a 6-part tie-breaking tuple:

1. I_{pinned} : A binary flag (1 or 0) indicating whether the item was manually pinned by an administrator.
2. I_{exact} : A binary flag (1 or 0) indicating if the query was an exact match to a keyword.
3. $\max(|k|)$: The length of the longest keyword that matched (favoring longer, more specific phrases).
4. $|K_j^{\text{match}}|$: The total number of individual keywords that matched the query.
5. $-\text{idx}(M_j)$: The negative index position in the configuration file (ensuring items listed earlier act as tie-breakers).
6. $\text{clicks}(M_j)$: Historical user click popularity.

A.3 Section 5: Approach 2 — LLM-as-Judge Classification

- **LLM Binary Relevance Indicator (relevant) — Equation 4:**

$$\text{relevant}(Q) = \mathbb{I}[\mathcal{L}_{\text{judge}}(\mathcal{T}_{\text{classify}}(Q)) = \text{YES}]$$

In English: Evaluates to 1 (True) if the Large Language Model $\mathcal{L}_{\text{judge}}$, given the formatted prompt template $\mathcal{T}_{\text{classify}}$, responds with “YES”, and 0 (False) otherwise.

- **LLM-Filtered Retrieval ($R_{\text{llm-judge}}$) — Equation 5:**

$$R_{\text{llm-judge}}(Q) = \begin{cases} R_{\text{kw}}(Q) & \text{if relevant}(Q) \\ \emptyset & \text{otherwise} \end{cases}$$

In English: If the LLM classifies the query as relevant to the company domain, the system returns the standard keyword candidate set $R_{\text{kw}}(Q)$. If classified as general knowledge, it returns an empty set $\emptyset$, suppressing all media.

A.4 Section 6: Approach 3 — LangChain Semantic Router

- **Route Vector Embedding — Equation 6:**

$$r_i = \mathcal{E}(\text{desc}_i), \quad \mathbf{q} = \mathcal{E}(Q)$$

In English: Converts both the user’s query Q and each candidate route description desc_i into high-dimensional vector representations ($\mathbf{q}$ and r_i) using an embedding function $\mathcal{E}$.

- **Route Selection via Cosine Similarity — Equation 7:**

$$\text{route}(Q) = \arg \max_i \frac{\mathbf{q} \cdot r_i}{\|\mathbf{q}\| \|r_i\|}$$

In English: Calculates the cosine similarity (the normalized dot product measuring geometric alignment in vector space) between the query vector $\mathbf{q}$ and every route vector r_i , selecting the index i of the route with the highest similarity score.

A.5 Section 7: Approach 4 — Per-Media Cosine Similarity Embedding Filter (Deployed)

- **Media Text Descriptor Construction (T_j) — Equation 8:**

$$T_j = \bigoplus_{\ell \in \{\text{en-US, zh-HK, zh-CN}\}} \text{title}_\ell(M_j) \oplus \bigoplus_{k \in K_j, k \not\sim \text{regex}, |k| > 2} k$$

In English: Constructs a rich textual representation T_j for media item j by concatenating ($\bigoplus$) its titles across three languages (English, Traditional Chinese/Hong Kong, Simplified Chinese/Mainland) along with all descriptive keywords, excluding tiny noise tokens ($|k| \leq 2$) and regex parameters.

- **Pre-Computed Media Embedding ($\mathbf{m}_j$) — Equation 9:**

$$\mathbf{m}_j = \mathcal{E}(T_j), \quad \forall j \in \{1, \dots, 31\}$$

In English: Converts each of the 31 unique media descriptors T_j into a fixed 1024-dimensional normalized vector $\mathbf{m}_j$ using the BGE-M3 embedding model at application startup.

- **Query-to-Media Cosine Similarity (sim_{cos}) — Equation 10:**

$$\text{sim}_{\text{cos}}(\mathbf{q}, \mathbf{m}_j) = \frac{\mathbf{q} \cdot \mathbf{m}_j}{\|\mathbf{q}\| \|\mathbf{m}_j\|} = \frac{\sum_{i=1}^d q_i m_{j,i}}{\sqrt{\sum_{i=1}^d q_i^2} \sqrt{\sum_{i=1}^d m_{j,i}^2}}$$

In English: Measures the semantic alignment between the incoming query vector $\mathbf{q}$ (reused from RAG search) and the media asset vector $\mathbf{m}_j$. The formula divides the dot product of the two vectors by the product of their lengths (Euclidean norms). Since vectors are pre-normalized, this simplifies to a direct dot product.

- **Dynamic Effective Threshold Calculation (τ_{eff}) — Equation 11:**

$$\tau_{\text{eff}}(M_j) = \tau_{\text{base}} \cdot \lambda(\text{exact}_j, \text{count}_j)$$

In English: Scales the baseline semantic acceptance threshold ($\tau_{\text{base}} = 0.43$) down using a modulation factor λ , making it easier for candidates with strong keyword matches to pass.

- **Modulation Factor Scale (λ) — Equation 12:**

$$\lambda(\text{exact}, \text{count}) = \begin{cases} 0.70 & \text{if exact} = 1 \quad (\text{exact keyword match}) \\ 0.82 & \text{if count} \geq 3 \quad (3+ \text{ keywords matched}) \\ 0.88 & \text{if count} \geq 2 \quad (2 \text{ keywords matched}) \\ 0.97 & \text{if count} \leq 1 \quad (0-1 \text{ keywords matched}) \end{cases}$$

In English: Determines the exact scaling percentage for the effective threshold: exact matches lower the threshold to 70% of base ($\tau_{\text{eff}} \approx 0.301$); 3+ matched keywords lower it to 82% ($\tau_{\text{eff}} \approx 0.353$); 2 matched keywords lower it to 88% ($\tau_{\text{eff}} \approx 0.378$); zero or one keyword matches keep the bar at 97% of base ($\tau_{\text{eff}} \approx 0.417$), applying the strictest semantic requirement to guard against weak substring collisions.

- **Final Media Acceptance Criterion (accept) — Equation 13:**

$$\text{accept}(M_j) = \begin{cases} \text{True} & \text{if } M_j \text{ is pinned (user-locked)} \\ \text{True} & \text{if } \text{sim}_{\cos}(\mathbf{q}, \mathbf{m}_j) \geq \tau_{\text{eff}}(M_j) \\ \text{False} & \text{otherwise} \end{cases}$$

In English: A media asset M_j is approved for display if it is manually pinned by a user OR if its cosine similarity score meets or exceeds its dynamically calculated effective threshold τ_{eff} . Otherwise, it is rejected as a false positive.